```
const express = require("express");
const crypto = require("crypto");
const fs = require("fs");
const path = require("path");
const mongoose = require("mongoose");
const Razorpay = require("razorpay");
const Payment = require("../models/Payment");

const router = express.Router();

const DATA_DIR = path.join(__dirname, "../data");
const PAYMENTS_FILE = path.join(DATA_DIR, "payments.json");

// ========================================
// SUBSCRIPTION AMOUNT
// Fixed at ₹106 (Razorpay expects the amount in
// the smallest currency unit, i.e. paise, for INR).
// 106 INR = 10600 paise
// ========================================
const SUBSCRIPTION_AMOUNT_INR = 106;
const SUBSCRIPTION_AMOUNT_PAISE = SUBSCRIPTION_AMOUNT_INR * 100; // 10600
const CURRENCY = "INR";
const PLAN_NAME = "Construction X Subscription";

function ensureStorage() {
  if (!fs.existsSync(DATA_DIR)) fs.mkdirSync(DATA_DIR, { recursive: true });
  if (!fs.existsSync(PAYMENTS_FILE)) fs.writeFileSync(PAYMENTS_FILE, "[]",
"utf8");
}

function readLocalPayments() {
  ensureStorage();
  try {
    return JSON.parse(fs.readFileSync(PAYMENTS_FILE, "utf8"));
  } catch {
    fs.writeFileSync(PAYMENTS_FILE, "[]", "utf8");
    return [];
  }
}

function writeLocalPayments(payments) {
  ensureStorage();
  fs.writeFileSync(PAYMENTS_FILE, JSON.stringify(payments, null, 2), "utf8");
}
```

```
function mongoAvailable() {
  return mongoose.connection.readyState === 1;
}

 function getRazorpayInstance() {
  const keyId = process.env.RAZORPAY_KEY_ID || "rzp_test_TfBBkdlzTYUo4M";
  const keySecret = process.env.RAZORPAY_KEY_SECRET ||
"zwo3l6cyWXMirqCf07J2Zc2R";

  if (!keyId || !keySecret) {
    return null;
  }

  return new Razorpay({ key_id: keyId, key_secret: keySecret });
}

// ==========================================
// POST /api/payment/create-order
// Creates a Razorpay order for the fixed ₹106
// subscription and stores a "created" record.
// ==========================================
router.post("/create-order", async (req, res) => {
  try {
    const razorpay = getRazorpayInstance();

    if (!razorpay) {
      return res.status(500).json({
        success: false,
        message:
          "Payment gateway is not configured. Set RAZORPAY_KEY_ID and
RAZORPAY_KEY_SECRET in server/.env."
      });
    }

    const email = typeof req.body?.email === "string" ?
req.body.email.trim().toLowerCase() : "";

    const options = {
      amount: SUBSCRIPTION_AMOUNT_PAISE, // 10600 paise = ₹106
      currency: CURRENCY,
      receipt: `receipt_${Date.now()}`,
      payment_capture: 1,
      notes: {
        plan: PLAN_NAME,
        email: email || "guest"
```

```
    }
  };

  const order = await razorpay.orders.create(options);

  const paymentRecord = {
    userEmail: email || "guest",
    razorpayOrderId: order.id,
    razorpayPaymentId: null,
    razorpaySignature: null,
    amount: SUBSCRIPTION_AMOUNT_PAISE,
    currency: CURRENCY,
    plan: PLAN_NAME,
    status: "created"
  };

  if (mongoAvailable()) {
    await Payment.create(paymentRecord);
  } else {
    const payments = readLocalPayments();
    payments.unshift({
      id: `local-${Date.now()}-${Math.random().toString(36).slice(2, 8)}`,
      ...paymentRecord,
      createdAt: new Date().toISOString()
    });
    writeLocalPayments(payments);
  }

  return res.status(201).json({
    success: true,
    order: {
      id: order.id,
      amount: order.amount,
      currency: order.currency
    },
    keyId: process.env.RAZORPAY_KEY_ID,
    amountInRupees: SUBSCRIPTION_AMOUNT_INR
  });
} catch (error) {
  console.error("CREATE ORDER ERROR:", error?.error || error);
  return res.status(500).json({
    success: false,
    message:
      (error && error.error && error.error.description) ||
      "Unable to create payment order. Please try again."
```

```
    });
  }
});

// ========================================
// POST /api/payment/verify
// Verifies the Razorpay payment signature and
// marks the subscription as paid.
// ========================================
router.post("/verify", async (req, res) => {
  try {
    const {
      razorpay_order_id,
      razorpay_payment_id,
      razorpay_signature,
      email
    } = req.body || {};

    if (!razorpay_order_id || !razorpay_payment_id || !razorpay_signature) {
      return res.status(400).json({
        success: false,
        message: "Missing payment verification details"
      });
    }

    const keySecret = process.env.RAZORPAY_KEY_SECRET;
    if (!keySecret) {
      return res.status(500).json({
        success: false,
        message: "Payment gateway is not configured on the server."
      });
    }

    const expectedSignature = crypto
      .createHmac("sha256", keySecret)
      .update(`${razorpay_order_id}|${razorpay_payment_id}`)
      .digest("hex");

    const isValid = expectedSignature === razorpay_signature;

    if (!isValid) {
      if (mongoAvailable()) {
        await Payment.findOneAndUpdate(
          { razorpayOrderId: razorpay_order_id },
          { status: "failed", razorpayPaymentId: razorpay_payment_id }
```

```
        );
      } else {
        const payments = readLocalPayments();
        const record = payments.find((p) => p.razorpayOrderId ===
razorpay_order_id);
        if (record) {
          record.status = "failed";
          record.razorpayPaymentId = razorpay_payment_id;
          writeLocalPayments(payments);
        }
      }

      return res.status(400).json({
        success: false,
        message: "Payment verification failed. Signature mismatch."
      });
    }

    const cleanEmail = typeof email === "string" ? email.trim().toLowerCase() :
"";

    if (mongoAvailable()) {
      await Payment.findOneAndUpdate(
        { razorpayOrderId: razorpay_order_id },
        {
          status: "paid",
          razorpayPaymentId: razorpay_payment_id,
          razorpaySignature: razorpay_signature,
          ...(cleanEmail ? { userEmail: cleanEmail } : {})
        }
      );
    } else {
      const payments = readLocalPayments();
      const record = payments.find((p) => p.razorpayOrderId ===
razorpay_order_id);
      if (record) {
        record.status = "paid";
        record.razorpayPaymentId = razorpay_payment_id;
        record.razorpaySignature = razorpay_signature;
        if (cleanEmail) record.userEmail = cleanEmail;
        writeLocalPayments(payments);
      }
    }

    return res.status(200).json({
```

```
      success: true,
      message: "Payment Successful! Subscription activated.",
      amountInRupees: SUBSCRIPTION_AMOUNT_INR
    });
  } catch (error) {
    console.error("VERIFY PAYMENT ERROR:", error);
    return res.status(500).json({
      success: false,
      message: "Payment verification failed. Please try again."
    });
  }
});

// ========================================
// GET /api/payment/status/:email
// Returns whether the given user has an active
// (paid) subscription. Used to restore the
// "Subscribed" badge after a page refresh.
// ========================================
router.get("/status/:email", async (req, res) => {
  try {
    const email = typeof req.params.email === "string" ?
req.params.email.trim().toLowerCase() : "";

    if (!email) {
      return res.status(400).json({ success: false, message: "Email is required"
});
    }

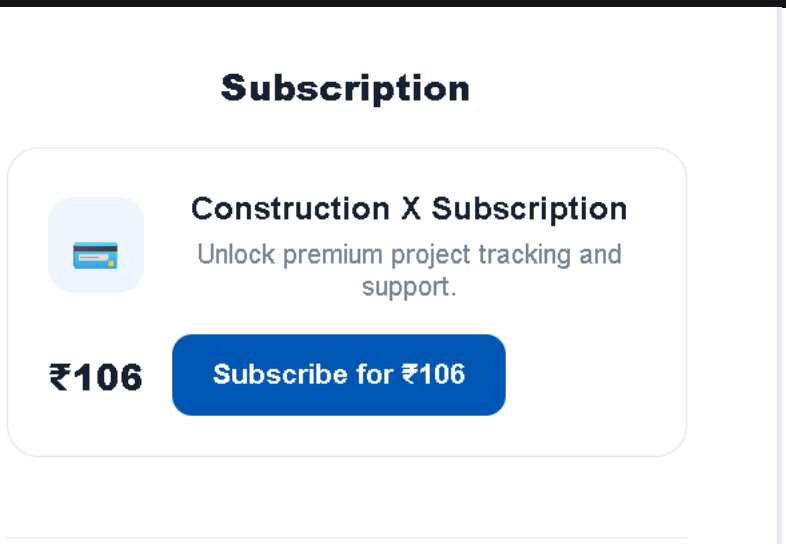
    let hasActiveSubscription = false;

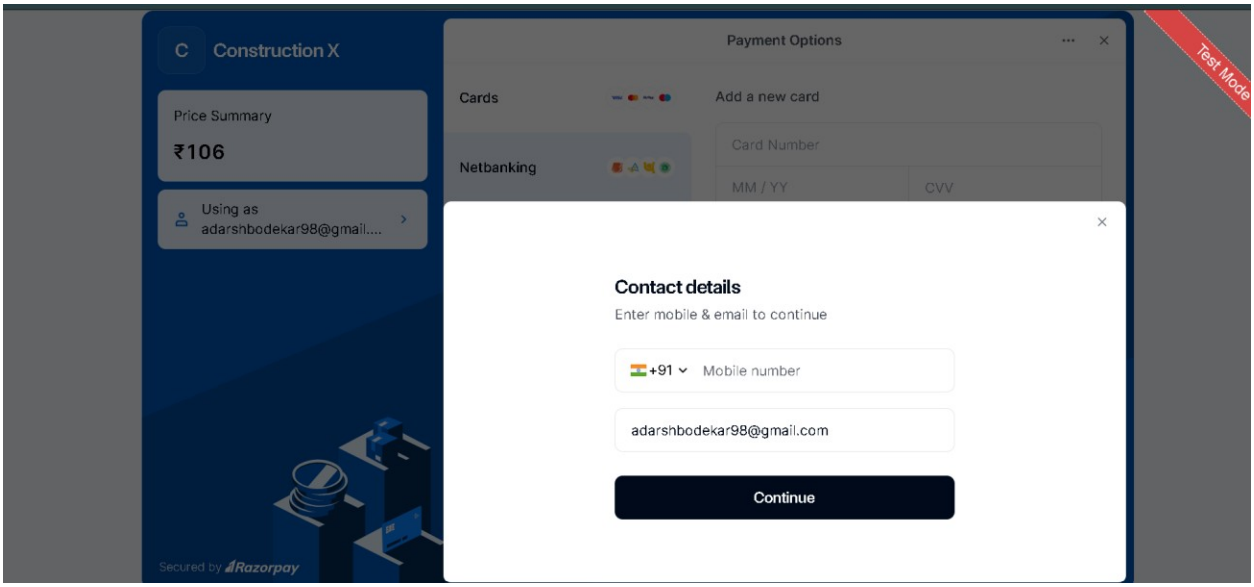
    if (mongoAvailable()) {
      const paid = await Payment.findOne({ userEmail: email, status: "paid" });
      hasActiveSubscription = !!paid;
    } else {
      const payments = readLocalPayments();
      hasActiveSubscription = payments.some(
        (p) => p.userEmail === email && p.status === "paid"
      );
    }

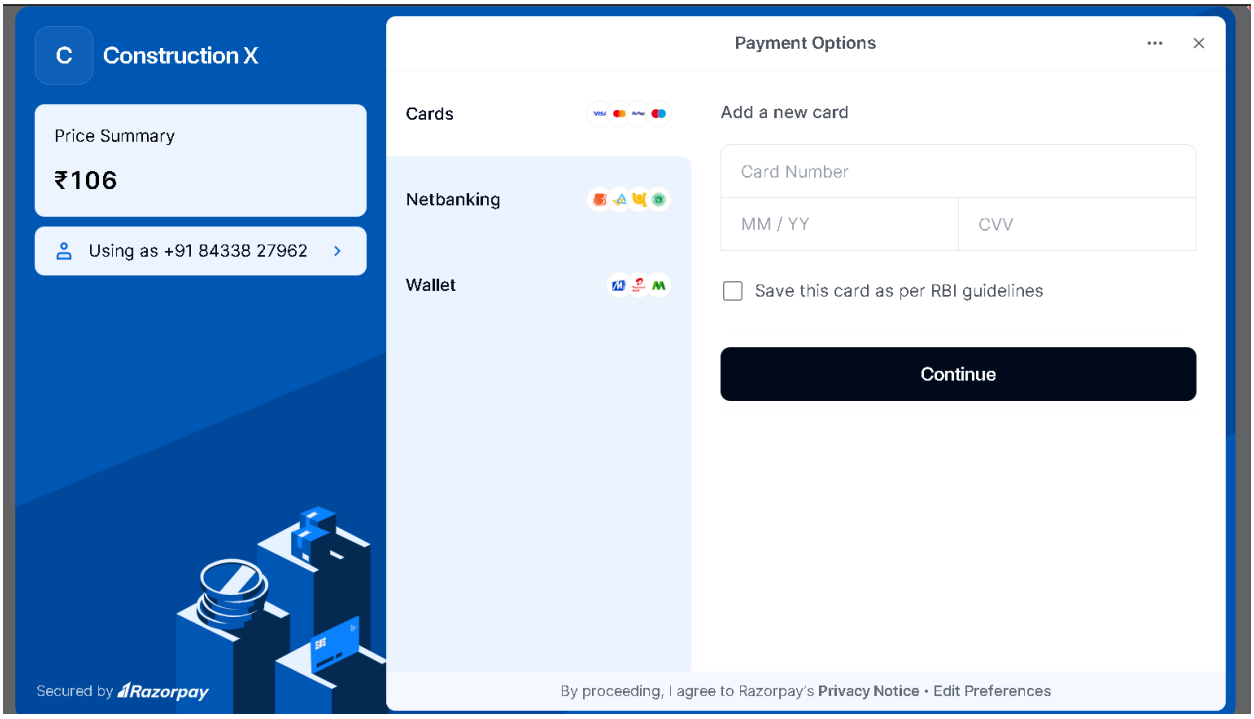
    return res.status(200).json({ success: true, subscribed:
hasActiveSubscription });
  } catch (error) {
    console.error("PAYMENT STATUS ERROR:", error);
```

```
    return res.status(500).json({ success: false, message: "Error checking
subscription status" });
  }
});

module.exports = router;
```




C
Construction X
Price Summary
₹106
Using as adarshbodekar98@gmail....
Payment Options
Cards
Add a new card
Netbanking
Card Number
MM / YY
CVV
Test Mode
Contact details
Enter mobile & email to continue
+91
Mobile number
adarshbodekar98@gmail.com
Continue
Secured by Razorpay


C
Construction X
Price Summary
₹106
Using as +91 84338 27962
Payment Options
Cards
Netbanking
Wallet
Add a new card
Card Number
MM / YY
CVV
Save this card as per RBI guidelines
Continue
Secured by Razorpay
By proceeding, I agree to Razorpay's Privacy Notice • Edit Preferences